

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

PERSONALIZED LEARNING RECOMMENDATION SYSTEM

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
24103460081	24103002862	BWU/BCA/ 24/084	Anish Dey

in partial fulfillment for the award of the degree

of

BACHELOR OF COMPUTER APPLICATIONS (HONS)



DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125
November 2026

PERSONALIZED LEARNING RECOMMENDATION SYSTEM

Submitted by

Roll Number	Registration Number	Student Code	Student Name
24103460081	24103002862	BWU/BCA/ 24/084	Anish Dey

in partial fulfillment for the award of the degree

of

BACHELOR OF COMPUTER APPLICATIONS (HONS)

In

DEPARTMENT OF COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

DEPARTMENT OF COMPUTATIONAL SCIENCES

BONAFIDE CERTIFICATE

Certified that the project report titled "PERSONALIZED LEARNING RECOMMENDATION SYSTEM" is the Bonafide work of "ANISH DEY " (Roll Number: 24103460081; Student Code: BWU/BCA/ 24/084), carried out in partial fulfilment of the requirements for the Bachelor of Computer Applications (Hons) degree, under the supervision of the faculty member assigned by the Department of Computational Sciences.

SIGNATURE

DR. JAYANTA AICH
HEAD OF THE DEPARTMENT

COMPUTATIONAL SCIENCES
BRAINWARE UNVERSITY

SIGNATURE

DR. RAAHUL KUMAR GHOSH
ASSOCIATE PROFESSOR

COMPUTATIONAL SCIENCES
BRAINWARE UNVERSITY

ACKNOWLEDGEMENT PAGE

I express my sincere gratitude to Brainware University and the Department of Computational Sciences for providing the academic environment and resources required to undertake this Machine Learning project.

I am thankful to the faculty member assigned as my project supervisor for guidance, suggestions and feedback during the development of this project. I also acknowledge the support of the faculty members who have helped me build the programming and data-analysis knowledge applied in this work.

Finally, I thank my family and well-wishers for their encouragement and support.

ABSTRACT

This report presents Edu Recommend, a Python-based Personalized Learning Recommendation System developed for the Machine Learning PBL (BCA57205), Group GR-1. The application accepts a student's subject, previous marks, weekly study hours, preferred learning format and completed topics. It compares a student profile with topic metadata using a content-based approach. Topic descriptions are converted into bag-of-words vectors with scikit-learn Count Vectorizer, and cosine similarity is calculated using NumPy. Additional score adjustments consider performance-derived difficulty tier, preferred learning format and prerequisite status. The application ranks and displays up to three learning topics with explanations. Pandas is used for CSV data loading and preprocessing, while Matplotlib generates an EDA dashboard based on student-history records. The supplied functional test suite was executed and reported seven of seven tests passed. The current implementation is a content-based similarity recommender with rule-based score adjustments; it does not train a supervised prediction model, and the supplied files do not establish recommendation effectiveness using labelled learner outcomes.

TABLE OF CONTENTS

- Abstract
- List of Tables
- List of Figures
- CHAPTER 1: INTRODUCTION
- CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEM
- CHAPTER 3: PROPOSED SYSTEM AND METHODOLOGY
- CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION
- CHAPTER 5: TESTING, RESULTS AND EVALUATION
- CHAPTER 6: CONCLUSION AND FUTURE SCOPE

LIST OF TABLES

Table 3.1 Dataset files and purpose

Table 3.2 Main dataset fields

Table 5.1 Functional test results

LIST OF FIGURES

Figure 3.1 Recommendation workflow

Figure 4.1 Project module structure

Figure 4.2 EDA dashboard (supplied project output

CHAPTER 1: INTRODUCTION

Background

Digital learning environments provide students with access to many topics and learning materials. However, students differ in their previous academic performance, preferred learning formats, subject interests and completed topics. A single fixed sequence of learning materials may therefore be unsuitable for every learner. A personalized learning recommendation system attempts to use a learner profile and information about available topics to suggest relevant next topics or resources.

- **1.2 Project Overview**

EduRecommend is a Python-based, terminal application for personalized learning-topic recommendations. The supplied implementation uses a topic catalogue and a student-history dataset. A student enters a subject, previous marks, weekly study hours, preferred learning format and completed topics. The application prepares a student profile and ranks candidate topics from the selected subject. It also provides exploratory data analysis (EDA) charts and stores recommendation sessions in a CSV log.

- **1.3 Problem Statement**

Students may need different levels of difficulty and different learning formats. They may also have completed some topics but not others. A static recommendation list does not account for these differences. The project addresses the problem of selecting and ranking suitable learning topics using available topic metadata and an individual student's learning profile.

- **1.4 Objectives**

The objectives are: (1) maintain a structured catalogue of learning topics; (2) load and clean student and topic CSV data; (3) explore student-learning data through EDA; (4) represent topic metadata and student preferences as numerical vectors; (5) calculate similarity and rank candidate topics; (6) consider performance tier, preferred format and prerequisite information; and (7) demonstrate the application through a Python terminal interface.

- **1.5 Scope**

  The supplied catalogue covers Python, Data Structures, DBMS and Web Development. The application is a local Python console program. It supports profile input, topic recommendations, EDA chart generation and CSV session logging. It does not contain a web interface, user authentication, a remote database or cloud deployment. The supplied code does not establish that the datasets represent a real institutional student population; their provenance should be confirmed before making claims about real-world representativeness.

- **1.6 Limitations**

  The current recommendation query uses subject, performance-derived difficulty tier and learning style. Although weekly study hours are collected, they are not included in the query vector or final recommendation score in the inspected recommender implementation. The historical student dataset is used for EDA and is not used to train a predictive model. The current implementation is a content-based similarity recommender with additional pedagogical rules; it is not a supervised model trained on labelled recommendation outcomes.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SYSTEM

- **2.1 Existing Learning Approaches**

  A fixed curriculum presents the same topic order to all learners. This is simple to administer but does not automatically adapt to a learner's performance or preferred format. Content-based recommendation approaches instead compare item attributes with a user profile. Collaborative filtering approaches use interactions or ratings from multiple users, but require sufficient interaction history. These are general descriptions of recommendation approaches; the supplied project archive does not include a formal literature survey or source citations for a particular deployed system.

- **2.2 Proposed Approach in Context**

  The project uses a content-based approach because the supplied topic catalogue contains descriptive attributes such as subject, difficulty, format and tags. A student profile can be compared with topic metadata without requiring a large student-topic rating matrix. This makes the method understandable and suitable for a small syllabus-oriented demonstration.

- **2.3 Limitations of Existing Approaches**

  A fixed topic sequence may not reflect individual readiness. Collaborative filtering may have a cold-start problem when interaction data is limited. Complex deep-learning recommenders can require more data and computation and may be harder to explain. The current project chooses a simpler and more interpretable content-based method; this is a design rationale rather than a measured comparison against those systems.

CHAPTER 3: PROPOSED SYSTEM AND METHODOLOGY

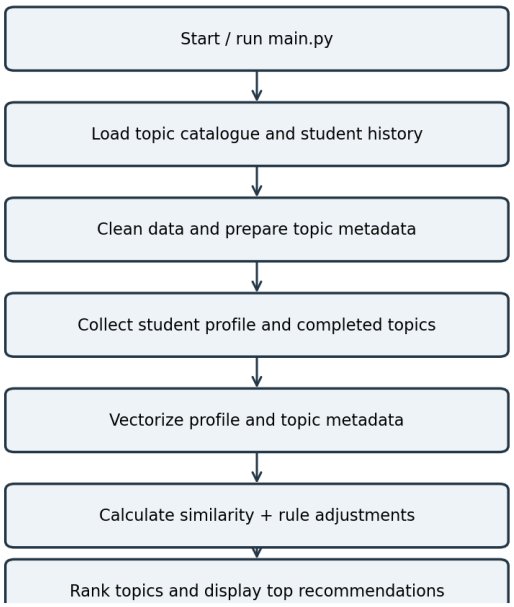
- **3.1 Proposed System**

The proposed system accepts an individual learner profile and compares it with available topics in the selected subject. Completed topics are excluded. The system creates text vectors from topic metadata and a student query, computes cosine similarity, applies additional score adjustments for difficulty and format matching, considers prerequisite status, and sorts candidates by final relevance score.

➢ **Table 3.1 Dataset files and purpose**

Dataset file	Purpose	Use in the system
topics_catalog.csv	Catalogue of available learning topics	Provides subject, topic, difficulty, format, estimated hours, prerequisites and tags
student_history.csv	Sample historical student records	Used for exploratory data analysis; it is not used to train a supervised model
user_logs.csv	Application interaction log	Stores recommendation-session details entered during application use

➢ **Figure 3.1 Recommendation workflow**



- **3.2 Tools and Technologies**

The inspected requirements file lists Pandas, NumPy, Matplotlib and scikit-learn. Python is the programming language and Visual Studio Code is the stated development environment. Count Vectorizer from scikit-learn is used to convert text metadata into bag-of-words vectors. No database or web framework is used in the supplied project.

➢ **Table 3.2 Main dataset fields**

Field	Meaning	Role
subject	Academic subject associated with a topic or learner	Subject matching
prior_marks	Learner's previous marks	Helps infer a difficulty tier
weekly_study_hours	Approximate study time per week	Collected as profile information; not currently part of similarity scoring
learning_style / primary_format	Preferred learning format and topic format	Format preference adjustment
difficulty_level	Difficulty assigned to a topic	Difficulty matching
prerequisite_topic	Topic recommended as a prerequisite	Prerequisite penalty when not completed
completion_rate	Historical completion percentage	Available in student-history data for analysis

- **3.3 Dataset Description**

The archive contains data/topics_catalog.csv and data/student_history.csv. The project documentation describes the topic catalogue as containing 25 curriculum items and the student-history file as containing 50 student records. The topic fields are topic_id, subject, topic_name, difficulty_level, primary_format, est_hours_required, prerequisite_topic and tags. The history fields are student_id, student_name, subject, prior_marks, weekly_study_hours, learning_style, current_difficulty, completion_rate and grade. A third file, data/user_logs.csv, stores example session records. The dataset source or collection method is not identified in the supplied files; this must be stated transparently in the final submission.

- **3.4 Data Preprocessing**

The DataProcessor class reads the topic catalogue and student history using Pandas. It strips whitespace from selected text fields, fills missing prerequisite and tag values, converts estimated hours, marks and weekly study hours to numeric values, and fills conversion failures with defaults. It then creates a metadata_text field by joining the lower-case subject, difficulty, primary format and tags. These operations are implemented in src/data_processor.py.

- **3.5 Feature Preparation**

  The topic metadata_text field is used as the text representation of each catalogue item. The student query is assembled from the selected subject, the difficulty tier inferred from marks and the selected learning style. CountVectorizer converts the topic text into a bag-of-words matrix and transforms the student query into a vector using the same vocabulary.

- **3.6 Recommendation Algorithm**

  First, candidate topics are filtered to the selected subject. The system then removes topics already listed as completed. For each remaining topic, cosine similarity is calculated between the student query vector and the topic vector. The score receives a 0.15 addition when topic difficulty matches the student's performance tier and a 0.10 addition when the topic format matches the preferred learning style. A 0.20 deduction is applied when the prerequisite is not marked completed. The score is converted to a displayed percentage bounded between 10 and 99, then candidates are sorted in descending order and the top three are returned.

- **3.7 Cosine Similarity**

  Cosine similarity measures the angle-based similarity between two vectors. The implementation uses the formula: similarity (A,B) = (A · B) / (||A|| × ||B||). The code uses NumPy dot product and vector norms. If either vector has zero norm, the function returns 0.0 to avoid division by zero. The displayed relevance percentage is a project-specific ranking score after rule adjustments; it should not be described as a calibrated probability or verified prediction accuracy.

- **3.8 Methodology Workflow**

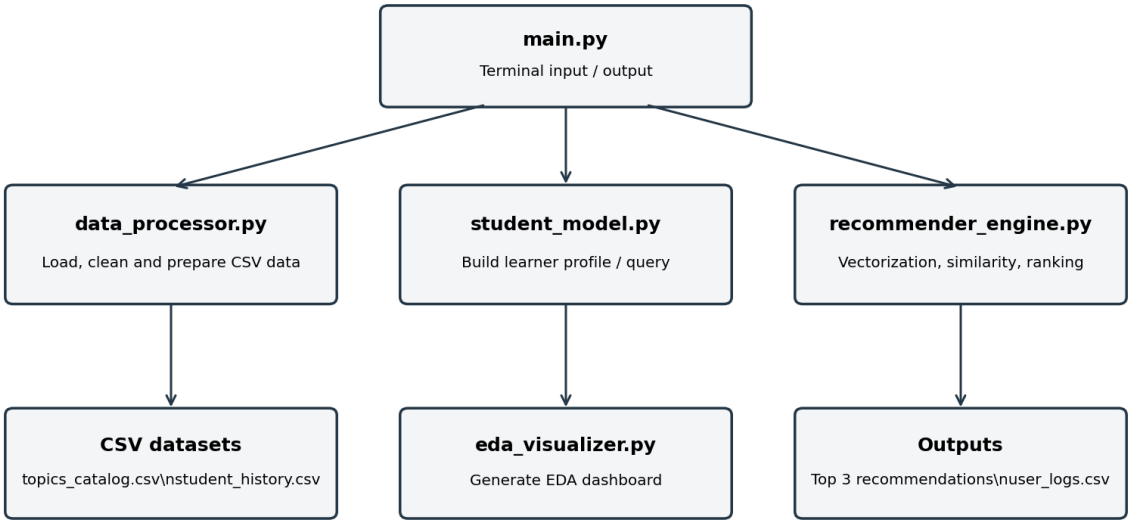
  The workflow is: load CSV files; clean and prepare data; create topic metadata; collect student profile; infer a difficulty tier from marks; filter topics by subject; exclude completed topics; vectorize topic metadata and student query; calculate cosine similarity; adjust scores using difficulty, format and prerequisite rules; sort recommendations; display the top recommendations; and optionally save the session to a CSV log. EDA is generated from the student-history dataset.

CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION

- **4.1 System Architecture**

The system consists of a terminal interface (main.py), a data-processing module (src/data_processor.py), a student profile model (src/student_model.py), a recommendation engine (src/recommender_engine.py), an EDA visualizer (src/eda_visualizer.py), CSV datasets under data/, and chart output under plots/. The modules are coordinated by the main application.

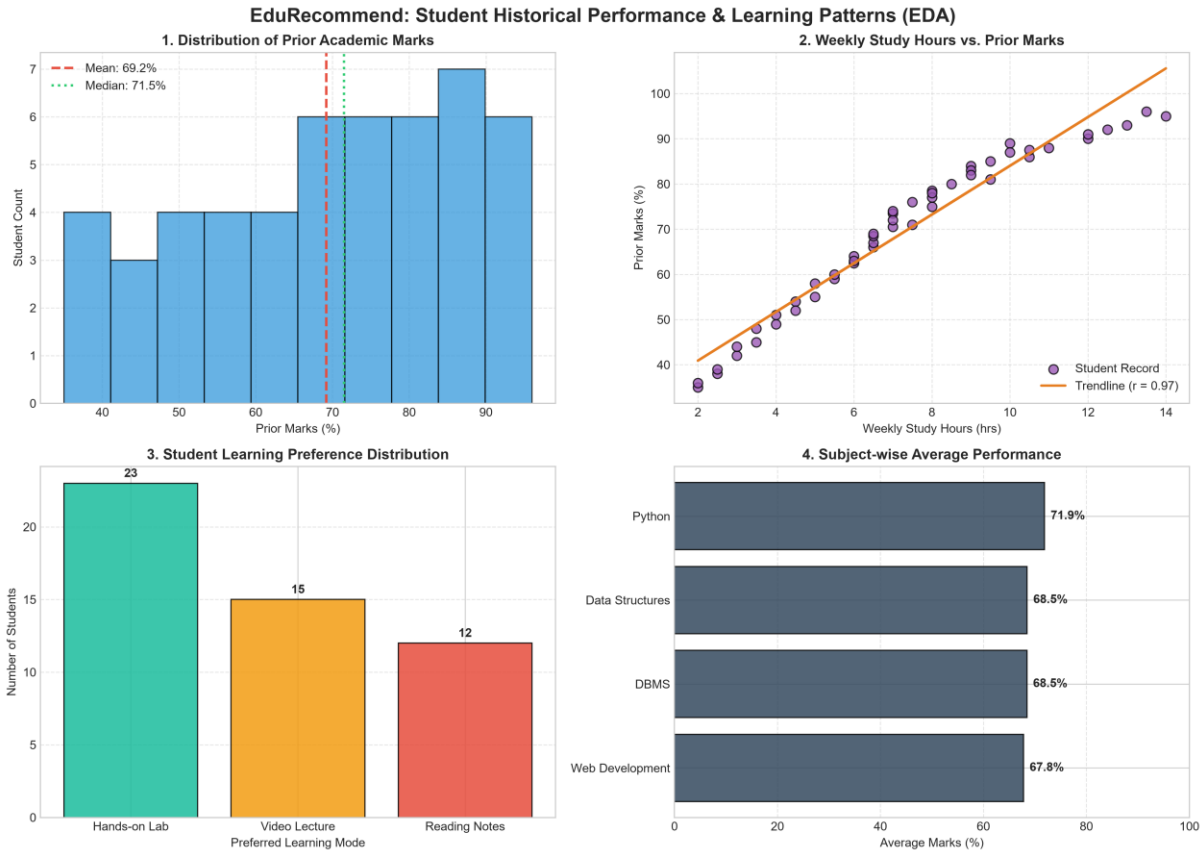
➢ **Figure 4.1 Project module structure**



- **4.2 Module Description**

Main.py provides the menu, validates user input, creates Student objects, calls the recommender, displays results, writes recommendation logs and invokes EDA generation. data_processor.py loads and cleans CSV data and creates topic metadata. student_model.py defines the Student class and validates name, subject, marks and study hours. recommender_engine.py implements vectorization, cosine similarity, score adjustments and ranking. eda_visualizer.py creates a four-panel Matplotlib dashboard.

➢ Figure 4.2 Exploratory data analysis dashboard



The EDA dashboard is the supplied project output. It visualizes marks distribution, weekly study hours versus marks, learning-format preferences and average marks by subject

- ## 4.3 Student Profile Input

The terminal flow requests the student's name, subject, previous marks, weekly study hours, preferred learning format and completed topics. Supported subjects in the Student class are Python, Data Structures, DBMS and Web Development. Supported learning formats are Hands-on Lab, Video Lecture and Reading Notes. Marks are validated within 0–100 and weekly study hours must be positive and no more than 80.

- ## 4.4 EDA and Visualization

The EDA visualizer creates a four-panel dashboard: distribution of prior marks, weekly study hours versus prior marks with a fitted trend line and correlation value, learning preference distribution, and subject-wise average marks. The generated image is saved as plots/eda_dashboard.png. The dashboard is based on student_history.csv.

- **4.5 File Handling**

  The application appends generated recommendation details to data/user_logs.csv. The log includes timestamp, student name, subject, marks, study hours, learning style, difficulty tier, recommendation rank, topic, format, relevance score and explanation. A menu option reads and displays recent log entries.

- **4.6 Python Concepts Demonstrated**

  Conditions and loops are used for input validation, filtering and menu navigation. Lists and dictionaries store completed topics and recommendation records. Functions divide the application into tasks. Sorting ranks recommendations. File handling reads and appends CSV files. Exception handling manages invalid inputs and file errors. Classes and objects represent students, data processing, recommendation and visualization. NumPy performs vector calculations and numerical analysis; Pandas handles tabular data; Matplotlib creates charts.

- **4.7 Internal Structure**

  Project files in the supplied archive include main.py; src/data_processor.py; src/student_model.py; src/recommender_engine.py;src/eda_visualizer.py; data/topics_catalog.csv; data/student_history.csv; data/user_logs.csv; plots/eda_dashboard.png; test_system.py; verify_datasets.py; README.md; PROJECT_REPORT.md; and requirements.txt.

CHAPTER 5: TESTING, RESULTS AND EVALUATION

- **5.1 Testing Approach**

The supplied test_system.py contains seven functional and edge-case checks. The test suite was executed against the uploaded archive in an isolated extracted workspace. It completed with a return code of 0 and reported 7/7 tests passed.

➢ **Table 5.1 Functional test results**

Test case	Expected behaviour	Result
Valid learner profile	Return up to three suitable recommendations	Passed
Marks above 100	Reject invalid marks	Passed
Negative weekly study hours	Reject invalid hours	Passed
Empty learner name	Reject empty name	Passed
Unsupported subject	Reject unsupported subject	Passed
All catalogue topics completed	Return no further recommendations	Passed
Zero-vector cosine similarity	Return 0.0 without a crash	Passed

- **5.2 Test Cases and Results**

Test 1: valid student profile and top-three recommendation generation — PASS. Test 2: marks above 100 rejected — PASS. Test 3: negative weekly study hours rejected — PASS. Test 4: empty student name rejected — PASS. Test 5: unsupported subject rejected — PASS. Test 6: no recommendations when all topics in a subject are marked completed — PASS. Test 7: zero-vector cosine similarity returns 0.0 without division by zero — PASS.

- **5.3 EDA Results**

The project includes a saved four-panel EDA image at plots/eda_dashboard.png. The charts describe marks distribution, the relationship between weekly study hours and marks, learning-style counts and subject-wise average marks. The actual values should be interpreted from the supplied dataset and chart; no causal relationship should be inferred from a scatter plot or correlation alone.

- **5.4 Recommendation Output**

  The application displays up to three ranked topics for a selected subject. Each recommendation includes topic name, difficulty level, primary format, estimated hours, displayed relevance score and a text explanation. The uploaded user_logs.csv contains example records for Anish Dey , including a Python profile with 76.5 prior marks and 8 weekly study hours. These are existing sample log entries, not proof that every profile or recommendation has been independently validated.

- **5.5 Evaluation Limitations**

  The supplied project does not implement a separate recommendation-effectiveness evaluation module with ground-truth relevance labels, held-out recommendation outcomes, Precision@K, MRR or Coverage calculations. The seven tests verify functional behaviour and edge cases, not recommendation accuracy or learning improvement. The final report therefore does not claim an accuracy percentage. A proper effectiveness evaluation would require labelled or observed relevant-topic outcomes and a defined evaluation protocol.

- **5.6 Observed Implementation Limitations**

  The recommender's student query contains subject, difficulty tier and learning style. Weekly study hours are collected and logged but are not used in the similarity query or score calculation. Student-history records support EDA but are not used to train a recommendation model. The current method is a content-based recommender using cosine similarity and rule-based adjustments, rather than a trained supervised machine-learning model. Prerequisite status affects ranking through a penalty; it does not prevent a topic with an incomplete prerequisite from being recommended.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

- **6.1 Conclusion**

EduRecommend demonstrates a local Python-based personalized learning-topic recommendation workflow. It uses structured topic metadata, student profile input, text vectorization, cosine similarity, pedagogical score adjustments, sorting and explanatory output. The project also demonstrates data cleaning, EDA, CSV file handling, object-oriented programming and input validation. The included test suite passed all seven supplied functional tests when executed in the inspected workspace. The project meets the expected demonstration concept of accepting student information and returning personalized topic recommendations, while its effectiveness as a learning recommender has not yet been established through labelled outcome evaluation.

- **6.2 Future Scope**

Possible future work, if permitted, includes using study-hour information in the ranking method, collecting genuine learner feedback, defining ground-truth relevant topics, conducting a formal offline evaluation, improving prerequisite handling, validating the dataset's provenance and expanding the topic catalogue. These are future possibilities and are not claimed as implemented features.

1. scikit-learn documentation, CountVectorizer: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html
2. NumPy documentation, numpy.dot: https://numpy.org/doc/stable/reference/generated/numpy.dot.html
3. NumPy documentation, numpy.linalg.norm: https://numpy.org/doc/stable/reference/generated/numpy.linalg.norm.html
4. Pandas documentation: https://pandas.pydata.org/docs/
5. Matplotlib documentation: https://matplotlib.org/stable/contents.html
6. Python documentation: https://docs.python.org/3/
7. Project datasets and implementation files supplied in the EduRecommend project archive.

APPENDIX: PROJECT EXECUTION

Install the dependencies from the project directory using: python -m pip install -r requirements.txt. Start the application using: python main.py. Run the supplied functional test suite using: python test_system.py. Generate the EDA dashboard using: python -m src.eda_visualizer. These commands are based on the README and project files supplied in the archive.

REFERENCES

1. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
2. The pandas development team. pandas Documentation. https://pandas.pydata.org/docs/
3. NumPy Developers. NumPy Documentation. https://numpy.org/doc/
4. Matplotlib Development Team. Matplotlib Documentation. https://matplotlib.org/stable/
5. scikit-learn Developers. CountVectorizer Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html
6. scikit-learn Developers. Cosine Similarity Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html
7. Brainware University. Machine Learning PBL project instructions and GR-1 project description supplied by the course instructor.
8. EduRecommend project source code, topic catalogue, student-history dataset and project README supplied for this report.